

Guidelines for Writing, Executing, and Submitting MATLAB Code

4th Feb, 2024

Dr. Saif Ali Jasim Al Grait

Guidelines for Writing, Executing, and Submitting MATLAB Code

This document describes the guidelines --- rules & recommendations --- for writing MATLAB code. These guidelines are intended to help you write code that is easy to understand, easy to troubleshoot, and that gives clear results.

- Rules: follow these rules to receive full credit on your assignments:

- o Cells
- o Title cell
- o General Information cell
- o Parameter Definition cell(s)
- o In-line descriptions of parameters
- o Units of quantities
- o Plotting
- o “publish” code for submission

Recommendations: follow these recommendations if you like (do not impact grade):

- o Save as separate m-files
- o Clean Up cell
- o MATLAB vectorization
- o Separate cells for calculation and visualization (plotting)
- o Executing & troubleshooting code

Guidelines for Writing, Executing, and Submitting MATLAB Code

II. Rule: Cells

Cells are just sections of MATLAB code. To create a cell, use two percent signs in succession, like this:

```
%% Name of cell
```

Two rules for cell creation:

- 1) The name of the cell is written in the same line as the two percent signs %%.
- 2) There must be at least one space between “%%” and the name text

Here are some reasons to use cells:

- Cells make it easier to read, understand, and therefore troubleshoot code
- Cells can execute (run) independently, making it easier to test code
- Cells are used by the MATLAB “publish” command to create formatted sections in the published document and to create a Table of Contents

Use cell names that make sense to the problem at hand. Some examples of cell names used previously are:

- “Independent variables”
- “Electric field calculation”
- “Electric field plot”

Guidelines for Writing, Executing, and Submitting MATLAB Code

III. Rule: Title cell

Create a title at the top of your m-file to provide a description of your m-file. A title is created by creating a Title cell. This is how to create a title cell:

- 1) write the title name next to the “%%” at the top of your mfile
- 2) do not write any descriptive text using “%” directly under the title name line

An example of a title for a homework assignment is:

```
%% HW2 P12: Temporal Phase phi(t)
```

This example uses the following naming syntax:

```
%% HW# P#: brief description of problem
```

where "HW" stands for "homework" and "P" stands for "problem." For class homework assignments, this naming convention is required.

An example title for a research report is:

```
%% Dispersion Impairments for Optical QPSK
```

Guidelines for Writing, Executing, and Submitting MATLAB Code

IV. Rule: General Information cell

After the title cell, create a cell for General Information in order to provide some basic information on your code. A General Information cell is created by:

- 1) writing “%% General Information”
- 2) skip a line
- 3) providing multiple lines, each with “%” following by descriptive text

For homework assignments, create a cell with the following information:

```
%% General Information
%
% Author: {give your name, or team-member names if team assignment}
%
% Date Created: {date}
%
% Context: {course number}, {course name}
%
% Objective: {description}
```

An example for homework assignments is:

```
%% General Information
%
% Authors: Claudio Ippolito & Jose Betances, Team Rocket
%
% Date Created: October 2013
%
% Context: TCET-740, Fiber-Optics Telecommunications Technology
%
% Objective: Plots BER vs Received Power for many cases
```

Recommendation: Clean Up Cell

After the General Information cell, consider creating a cell that cleans up various things. Decide which things to clean up based on your needs.

Here is an example:

```
%% Clean Up  
clear all; % clears all of the Workspace variables  
clc; % clears the command window  
close all; % clears all figure windows
```

Recommendation: Clean Up Cell

After the General Information cell, consider creating a cell that cleans up various things. Decide which things to clean up based on your needs.

Here is an example:

```
%% Clean Up  
clear all; % clears all of the Workspace variables  
clc; % clears the command window  
close all; % clears all figure windows
```

Parameter Definitions

V. Rule: Parameter Definitions Cell(s)

Create at least one Cell that contains the parameters and variables. You might prefer to make multiple parameter cells to group similar/related quantities. Place the parameter cell (or cells) after the General Information cell, but near the beginning of your file. Call the cell “Parameter Definitions” and be more descriptive if you have more than one such cell.

Here is an example:

```
%% Parameter Definitions
CL1 = 0.9; % [--] confidence level
CL2 = 0.95; % [--] confidence level
CL3 = 0.99; % [--] confidence level
Tmin = 0.1; % [s] minimum measurement time
Tmax = 10; % [s] minimum measurement time
N = 100; % [--] number of array points
T = linspace(Tmin, Tmax, N); % [s] time array
```

The parameter cell (or cells) should be a one-stop shop for all of your code’s mathematical quantities. Some advantages of grouping all such quantities together are:

- a) you don’t need to search through the whole to find and edit a quantity value
- b) you can more readily compare quantity units to see if they are correct

In-line descriptions

VI. Rule: In-line descriptions

Write an in-line description for every quantity (e.g., parameter, variable, etc.) in your MATLAB code. These descriptions improve clarity, readability, and the ease of troubleshooting. To write an in-line description, use one percent sign “%” followed by the description, such as:

```
A = 2; % [sqrt(W)] amplitude
w = 2 * pi * 200; % [rad-THz] angular frequency
n = 1.50; % [--] refractive index
tps = [ 1 : 1 : 100]; % [ps] time vector
E = A * exp(1i * w * t); % [sqrt(W)] electric field
```

As part of the description, you must include the units of the quantity being described. If the quantity doesn't have any units, use “[--]”. The use of units helps in keeping track of units, especially when quantities of various units are combined together to form other quantities. The use of units is discussed further in the rule below.

Units of quantities

VII. Rule: Units of quantities

In regards to units, it is important to:

- a) Think carefully about the units in your calculations so that the numerical values do not approach the numerical noise of your computer (in order to avoid numerical noise errors)
- b) Record units in your in-line descriptions (in order to be able to easily track them throughout your code and avoid mistakes)

Recording units (b) was first discussed in the rule above.

Let's say you want to create a time vector with values from 0 to 100 picoseconds (where a picosecond = $1\text{E}-12$ seconds). One way to code this vector is as follows:

```
ts = [0 : 0.01E-12 : 100E-12]; % time vector, [s]
```

The values included in the above time vector `ts` are a very natural way to program --- the values correspond to the SI unit of seconds [s]. To emphasize the units, the programmer has placed "[s]" as an in-line comment.

The problem with the above time vector `ts` is that its values are very small. Computers don't like very small numbers because computers have a *computational noise limit*. If the size of the number is around (or less than) this noise limit, watch out! Your computer can generate numerical noise that can mess up your calculations. Let's say your computer has a noise limit of $1\text{E}-16$; this number might seem small, but square any value of the above time vector, and the squared value has fallen below the noise limit.

Units of quantities

Back to the time vector `t` ... the goal is to create a time vector whose values vary from 0 to 100 picoseconds. Here is a better way to create that vector:

```
tps = [0 : 0.01 : 100]; % time vector, [ps]
```

Units of quantities

If you are uncomfortable not using SI units, here are some words to win you over: with only one exception, MATLAB *doesn't know units*! MATLAB itself doesn't associate any units with any of the following vectors:

```
ts = [0 : 0.01E-12 : 100E-12]; % time vector, [s]  
tps = [0 : 0.01 : 100]; % time vector, [ps]
```

Instead, the units are only in the mind of the programmer. That is why it is good practice to write an in-line comment such as “% [s]” or “% [ps]” so that you (as a programmer) can remember what the associated units are.

Plotting

VIII. Rule: Plotting

Graphs are an important means of communicating information and should be accurate and clear: Imagine that you are working at a company and you are presenting your graph to your boss. Better yet, imagine that you are working at a company and you are preparing your graph for your boss to present to his/her boss.

Follow these rules when preparing x-y plots in MATLAB:

1. Use a sufficient number of data points (if you are allowed in the problem-under-study to increase the number of data points) to make smooth functions appear smooth.
 - a. An excellent way to define a vector is to use the MATLAB “linspace” function. Here is an example that creates a vector from 3 to 20, with 1000 points (including the end points).

```
t = linspace(3, 20, 1000); % [s] time array
```

See MATLAB help for further details

Plotting

2. If your number of data points is restricted in some way such that (for a single data trace) there is 50 or less points, make sure the following is visible for each data trace:

- a. Use a marker of some kind on each data point. The following plot command places a blue “*” marker on every data point, and connects the data points with straight lines:

```
plot(t, y, '-b*');
```

- b. Use a *straight* line segment to connect each neighboring pair of data points

3. Make a title for the graph, using the MATLAB function “*title*”. Here is how to create a title:

```
title('Name of Title Here')
```

- a. For homework problems, use the title: “HW# P#”
- b. For research reports, consider stating the main parameter values that distinguish the graph from other graphs (such as distance or coupling coefficient).
- c. For figures having several subplots, use the MATLAB function “*suptitle*” to create a single title for the whole figure

4. Axes

- a. Correctly and clearly label each axis with a descriptive word or phrase
- b. Make sure the y-axis label appears along the left-hand side
- c. Make sure the x-axis label appears along the bottom of the graph

Plotting

- d. Correctly and clearly provide units as part of each axis label. If the quantity is unitless, use [--].
- e. Never use numeric values such that an axis is multiplied by an exponential, such as 10^{-6} . Exponential numbers are hard to read and also easy for readers to miss. Instead, scale your units such that the significant digit appears within 3 digits of the decimal point. For example, use 0.01 [mW] instead of 0.00001 [W]. As another example, use 10 [ns] instead of 10,000 [ps].
- f. Set the axis limits as directed (if directed); use the “ylim” and “xlim” MATLAB functions to set the limits of the y and x axes, respectively. The following example sets the y limits at 0 and 10 and the x limits at 2 and 4

```
xlim([2 4])  
ylim([0 10])
```

- g. Make sure the tick numbers are large enough to be legible
- h. The numbers along the x-axis should vary from lowest on the left to highest on the right
- i. The numbers along the y-axis should vary from lowest on the bottom to highest on the top

Plotting

5. Single graph with multiple curves

a. Label each curve separately, using one of the following methods:

- i. Make a legend, using the “legend” command. Make sure each line is distinguishable in some way (e.g., different markers, different line style. The following example plots and creates a legend for three lines, having different colors and markers:

```
plot(t, d1, '-ro', t, d2, '-b*', t, d3, '-kx')  
legend('d1' , 'd2' , 'd3')
```

- ii. Make a text box for each line, using the “text” command. This option is a little more involved/difficult than the legend-command option

6. Use the “grid on” function to place a grid on your graph

Recommendation: MATLAB vectorization

IX. Recommendation: MATLAB vectorization

When programming in MATLAB, vector quantities can be created using the following syntax:

```
f = 10:1:90; % vector
```

This expression creates a vector of points whose values range from 10 to 90 in steps of 1. Note that this expression does not use a loop, as in the following:

```
for n = 10 to 90
    f(n) = n; % vector
end
```

In MATLAB, you can often avoid loop structures.

Once a vector is created, it can be used to create new vectors. The following statements all create new vectors using the previously created vector `f` and without using loops:

```
g = 2 * f; % g vector
h = f.^2; % h vector, where each element is the square of f
v1 = cos(f); % cosine vector
```

Rule: “publish” code for submission

XII. Rule: “publish” code for submission

To prepare your code for submission, use the MATLAB *publish* command to create a pdf file of your code (with figures executed and included). Submit this pdf file.

There are two main ways to invoke the publish command:

- 1) Type the *publish* command in the MATLAB Command window, using the name of the m-file name and the desired output file format as the function arguments:

```
>> publish('mfile_name.m', 'pdf')
```

- 2) For recent versions of MATLAB, there is a “PUBLISH” menu as part of the MATLAB Editor:

- Select the “PUBLISH” tab (there are three tabs to choose from --- “EDITOR, PUBLISH, and VIEW”)
- At the far right, there is a Publish icon.
- Use the pull-down menu to Edit Configurations, such as *set the Output file format to pdf*; close menu when configurations are set.
- Press the icon to publish your code.

The “published” code file provides several advantages:

- Uses the first cell name to create a document title
- Uses the remaining cell names to create a Table of Contents
- Uses cell names to create sections within the published document
- Generates plots and places each plot where it is called in the code

Make sure you use “;” at the end of every line of code. This symbol suppresses the following:

- a) output of variable values to your command line
- b) output of variable values to into your published file

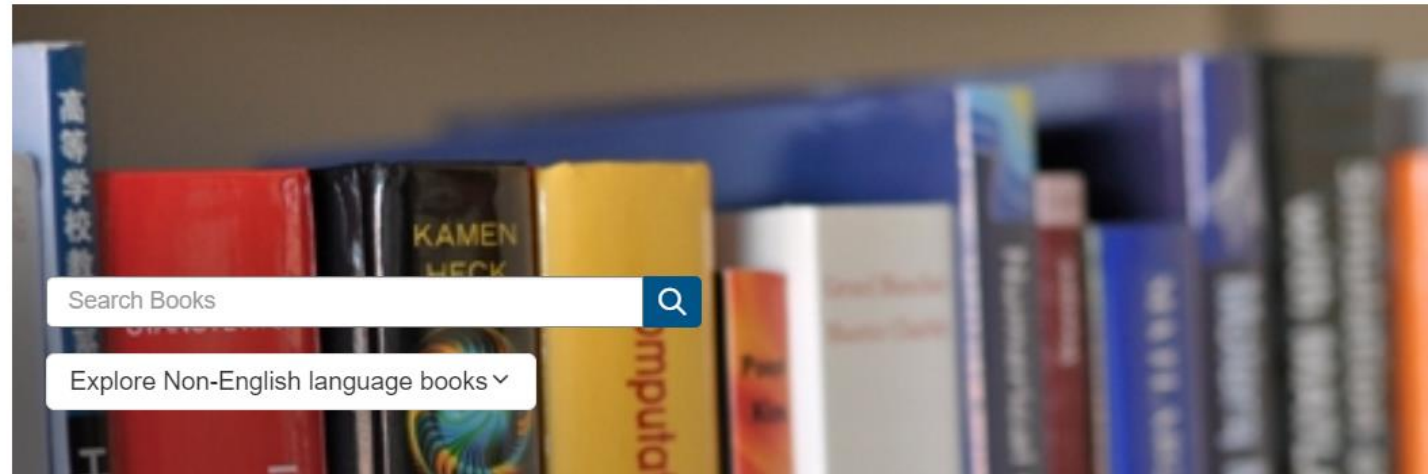
Online Resources

 [Products](#) [Solutions](#) [Academia](#) [Support](#) [Community](#) [Events](#) [Get MATLAB](#)  

MATLAB and Simulink Based Books

 Books ▾ 

[Books Main Page](#) | [Search](#) | [Join Book Program](#)



More than 2000 titles for teachers, students, and professionals

The ever-increasing number of books based on MathWorks products reflects the widespread use of these tools for research and development. The texts present theory, real-world examples, and exercises using MATLAB, Simulink, and other MathWorks products. They provide course materials for instructors in engineering, science, finance, and mathematics, and serve as authoritative references for researchers in academia and industry.

[Books - MATLAB & Simulink \(mathworks.com\)](https://www.mathworks.com/books)

Introduction to MATLAB - zyBook

This web-based book presents a comprehensive introduction to MATLAB through interactive questions, animations, and automated MATLAB assessment.

» [Learn more](#)

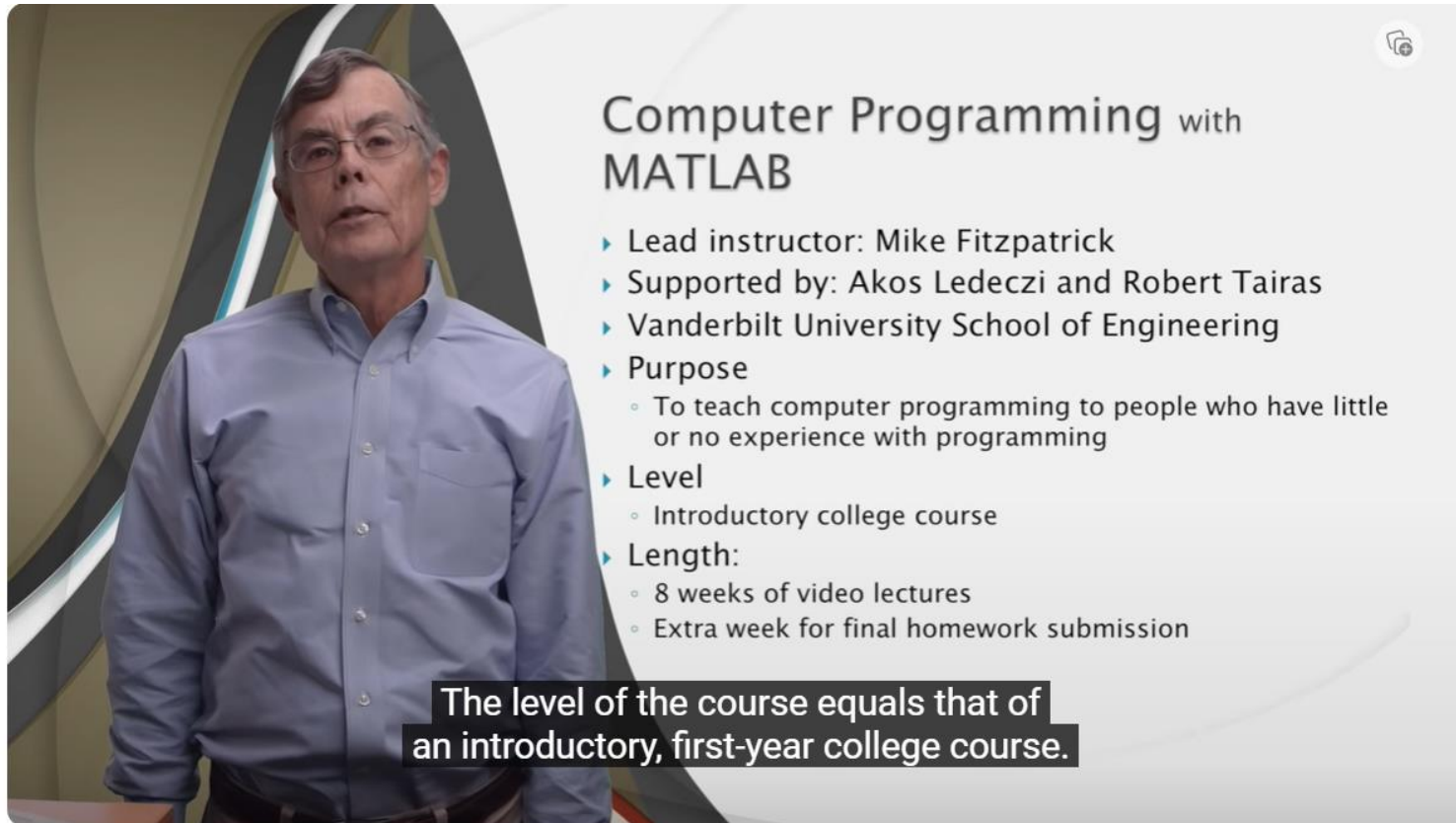
MATLAB Courseware

Teaching materials based on MATLAB and Simulink.

» [Find full courses and labs](#)

Online Resources-lectures on YouTube

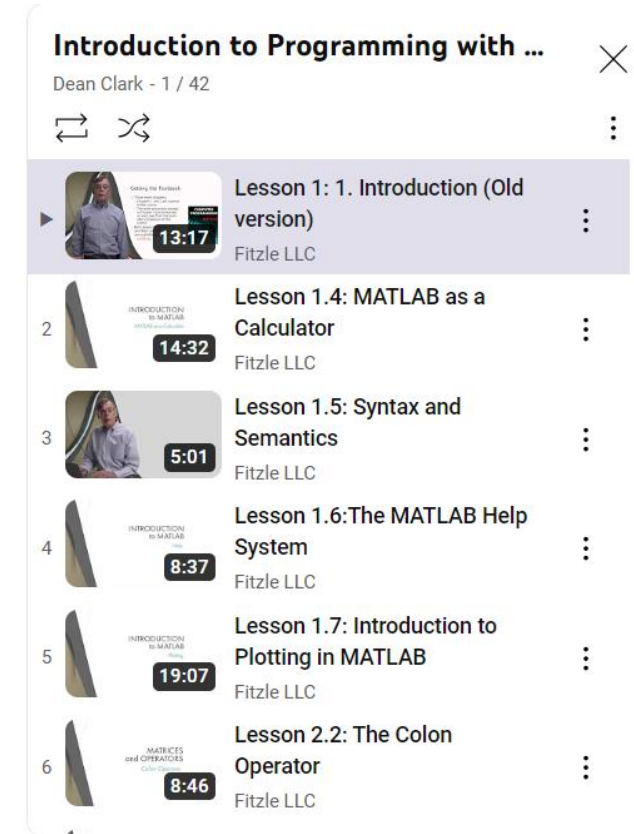
[Lesson 1: 1. Introduction \(Old version\) - YouTube](#)



Computer Programming with MATLAB

- ▶ Lead instructor: Mike Fitzpatrick
- ▶ Supported by: Akos Ledeczki and Robert Tairas
- ▶ Vanderbilt University School of Engineering
- ▶ Purpose
 - To teach computer programming to people who have little or no experience with programming
- ▶ Level
 - Introductory college course
- ▶ Length:
 - 8 weeks of video lectures
 - Extra week for final homework submission

The level of the course equals that of an introductory, first-year college course.



Introduction to Programming with MATLAB Dean Clark - 1 / 42

Lesson 1: 1. Introduction (Old version) 13:17 Fitze LLC

2 Lesson 1.4: MATLAB as a Calculator 14:32 Fitze LLC

3 Lesson 1.5: Syntax and Semantics 5:01 Fitze LLC

4 Lesson 1.6: The MATLAB Help System 8:37 Fitze LLC

5 Lesson 1.7: Introduction to Plotting in MATLAB 19:07 Fitze LLC

6 Lesson 2.2: The Colon Operator 8:46 Fitze LLC